Name: Kamran Gul

CMS: 023-25-0161

Section: C

OOP Lab: 11

Task 1

Shape and Rectangle (Abstract area)

Scenario:

A drawing app works with **shapes**. Every shape has a **color**, but **area** depends on the **kind** of shape (rectangle, circle, ...). The **general** idea "shape" should not invent a fake area; only **concrete** shapes provide a formula. Model **`Shape`** as **abstract** with an **abstract** `getArea()`. Implement **`Rectangle`**.

Requirements:

1. **Abstract class `Shape`:**
   - Private attribute: `color` (String).
   - Constructor: `Shape(String color)` — initializes `color`.
   - Getter: `getColor()`.
   - **Abstract method:** `public abstract double getArea();` — no body.
   - **Concrete method:** `display()` that prints the color and the area by calling `getColor()` and `getArea()` (so subclasses do not repeat this logic).
2. **Concrete class `Rectangle` extends `Shape`:**
   - Private attributes: `width` (double), `height` (double).
   - Constructor: `Rectangle(String color, double width, double height)` — first line `super(color);`, then set width and height.
   - **Implement** `getArea()` to return `width * height` (use `@Override`).
   - Add method `getPerimeter()` returning `2 * (width + height)` (not abstract).
3. **In `Main`:**
   - You **cannot** create `new Shape(...)` — comment in your report that this would not compile.
   - Create `Rectangle r = new Rectangle("Red", 4.0, 5.0);`
   - Call `r.display()` and print `r.getArea()` and `r.getPerimeter()` — expect area **20.0**, perimeter **18.0**.

Solution

```
abstract class Shape{
    private String color;
    Shape(String color){
        this.color = color;
    }
    String getColor(){
        return this.color;
    }
    public abstract double getArea();
    void display(){
        System.out.printf("Color: %s\n",getColor());
        System.out.printf("Area: %.2f\n",getArea());
    }
}
class Rectangle extends Shape{
    double width,height;
    Rectangle(String color,double width,double height){
        super(color);
        this.width = width;
        this.height = height;
    }
    @Override
    public double getArea(){
        return width * height;
    }
    double getPerimeter(){
        return 2 * (width + height);
   }
}

class Main {
    public static void main(String[] args) {
        // Shape shape = new Shape(); // This will not compile bcz we cannot have
an object of an abstract class
        Rectangle rectangle = new Rectangle("Red",2,3.4);
        rectangle.display();
        System.out.println("Area of Rectangle: " +  rectangle.getArea());
        System.out.println("Perimeter of Rectangle: "+rectangle.getPerimeter());
    }
}
```

Output

```
Color: Red
Area: 6.80
Area of Rectangle: 6.8
Perimeter of Rectangle: 10.8
```

Task 2

Employee and HourlyEmployee (Abstract pay)

Scenario:

HR software tracks **employees**. Every employee has a **name** and **ID**, but **monthly gross pay** is calculated **differently** (salary, hourly, commission). The **generic** "employee" should not invent a pay amount; only **concrete** employment types implement the rule. Model **`Employee`** as **abstract** with **abstract** `monthlyPay()`. Implement **`HourlyEmployee`** (paid by hours × rate).

Requirements:

1. **Abstract class `Employee`:**
   - Private attributes: `name` (String), `employeeId` (String).
   - Constructor: `Employee(String name, String employeeId)`.
   - Getters: `getName()`, `getEmployeeId()` (optional but useful).
   - **Abstract method:** `public abstract double monthlyPay();`
   - **Concrete method:** `display()` that prints name and ID.
2. **Concrete class `HourlyEmployee` extends `Employee`:**
   - Private attributes: `hoursWorked` (double), `hourlyRate` (double).
   - Constructor: `HourlyEmployee(String name, String employeeId, double hoursWorked, double hourlyRate)` — first line `super(name, employeeId);`.
   - **Implement** `monthlyPay()` to return `hoursWorked * hourlyRate` (document if you treat this as "this month" pay).
3. **In `Main`:**
   - Create `HourlyEmployee e = new HourlyEmployee("Ali", "E102", 80, 50);`
   - Print `e.monthlyPay()` (expect **4000.0**) and call `e.display()`.

Solution

```java
abstract class Employee{
      private String name;
      private String employeeId;
      Employee(String name,String employeeId){
            this.name= name;
            this.employeeId = employeeId;
      }
      public abstract double monthlyPay();

      void display(){
            System.out.printf("Name: %s\nEmployee Id: %s\n",name,employeeId);
      }
}
class HourlyEmployee extends Employee{
      private double hoursWorked;
      private double hourlyRate;
      HourlyEmployee(String name,String employeeId,double hoursWorked, double
hourlyRate){
            super(name,employeeId);
            this.hourlyRate = hourlyRate;
            this.hoursWorked = hoursWorked;
```

```
        }
        public double monthlyPay(){
                return hourlyRate * hoursWorked;
        }
}
class Main{
        public static void main(String[] args){
                HourlyEmployee e = new HourlyEmployee("Ali","E102",80,50);
                e.display();
                System.out.print("Monthly Pay: "+e.monthlyPay());
        }
}
```

Output

```
Name: Ali
Employee Id: E102
Monthly Pay: 4000.0
```

Task 3

Invoice and TaxedInvoice (Abstract amount due)

Scenario:

Billing uses **invoices**. Every invoice has an **invoice number** and a **base amount**, but the **final amount due** might add **tax** or other rules. The **abstract** invoice type forces each concrete invoice to say how much is **actually due**. Implement `**TaxedInvoice**` that adds a **tax percent** to the base.

Requirements:

1. **Abstract class `Invoice`:**
   - Private attributes: `invoiceNumber` (String), `baseAmount` (double).
   - Constructor: `Invoice(String invoiceNumber, double baseAmount)`.
   - Getter for `baseAmount` (e.g. `getBaseAmount()`) so subclasses can use it.
   - **Abstract method:** `public abstract double amountDue();` — final amount to pay for this invoice type.
   - **Concrete method:** `printHeader()` that prints invoice number and base amount.
2. **Concrete class `TaxedInvoice` extends `Invoice`:**
   - Private attribute: `taxPercent` (double) — e.g. `15` means 15%.
   - Constructor: `TaxedInvoice(String invoiceNumber, double baseAmount, double taxPercent)` — first line `super(invoiceNumber, baseAmount);`.
   - **Implement** `amountDue()` to return `getBaseAmount() * (1 + taxPercent / 100)`.
3. **In `Main`:**
   - Create `TaxedInvoice inv = new TaxedInvoice("INV-01", 1000, 15);`
   - Call `printHeader()` and print `amountDue()` — expect **1150.0** for 15% tax on 1000.

Solution

```
abstract class Invoice{
      private String invoiceNumber;
      private double baseAmount;
      Invoice(String invoiceNumber,double baseAmount){
            this.invoiceNumber = invoiceNumber;
            this.baseAmount = baseAmount;
      }
      double getBaseAmount(){
            return this.baseAmount;
      }
      public abstract double amountDue();
      void printHeader(){
            System.out.printf("Invoice Number: %s\nBase Amount:
%.2f\n",invoiceNumber,baseAmount);
      }
}
class TaxedInvoice extends Invoice{
      private double taxPercent;
      TaxedInvoice(String invoiceNumber,double baseAmount,double taxPercent){
            super(invoiceNumber,baseAmount);
            this.taxPercent = taxPercent;
      }
      public double amountDue(){
            return getBaseAmount() * (1 + taxPercent/100.0);
      }
}
class Main{
      public static void main(String[] args){
            TaxedInvoice inv = new TaxedInvoice("INV-01",1000,15);
            inv.printHeader();
            System.out.print(inv.amountDue());
      }
}
```

Output

```
Invoice Number: INV-01
Base Amount: 1000.00
1150.0
```